



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Java Database Connectivity



Java Database Connectivity (JDBC)

Introduction

- **JDBC** (Java Database Connectivity) is a Java-based data access technology from Oracle Corporation.
- JDBC is a **Java API** (Application Programming Interface). It provides methods for querying and updating data in a database.
- It is a part of **JavaSE** (Java Standard Edition).
- JDBC was initially conceived as a **client-side API**, enabling a Java client to interact with a data source
- JDBC release then has featured updates to both the **client-side package (java.sql)** and the **server-side package (javax.sql)**

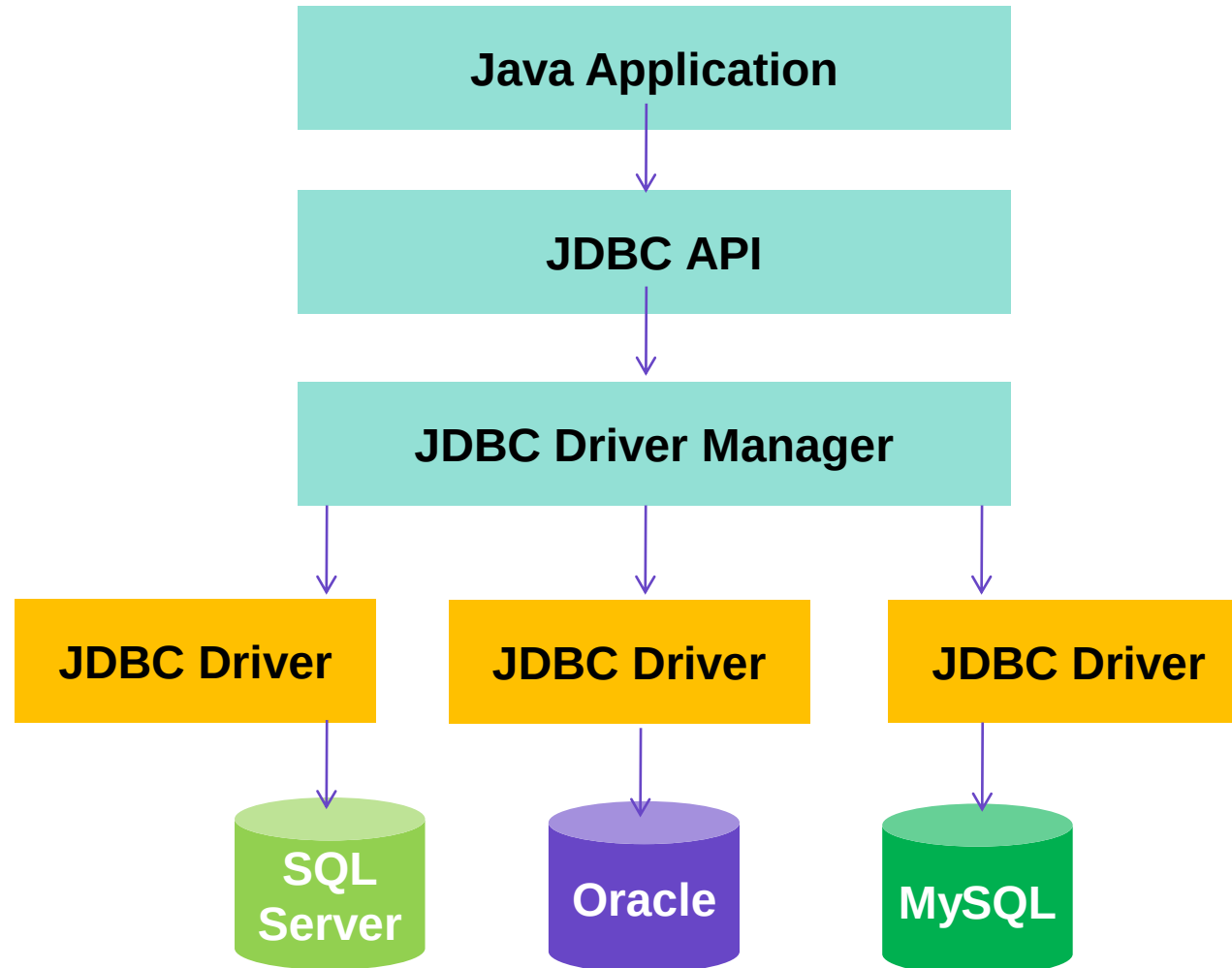
Java Database Connectivity (JDBC)

Why JDBC ?

- Before JDBC, **ODBC API** was the **database API** to connect and execute the query with the database.
- But, ODBC API uses **ODBC driver** which is written in **C language** (i.e. platform dependent and unsecured).
- That is why Java has defined its **own API** (JDBC API) that uses JDBC drivers which is written in Java language.
- We can use JDBC API to handle database using **Java program**.

Java Database Connectivity (JDBC)

JDBC Architecture



Java Database Connectivity (JDBC)

JDBC Architecture

- JDBC offers a **programming-level interface** that handles the mechanics of Java applications communicating with a **database or RDBMS**.
- JDBC interface consists of **two layers**:
- **JDBC API** supports communication between the **Java application** and the **JDBC driver manager**.
- JDBC driver supports communication between the **JDBC driver manager** and the **database driver**.
- **JDBC Driver Manager**: The basic service for managing a set of JDBC drivers.

Java Database Connectivity (JDBC)

JDBC Drivers

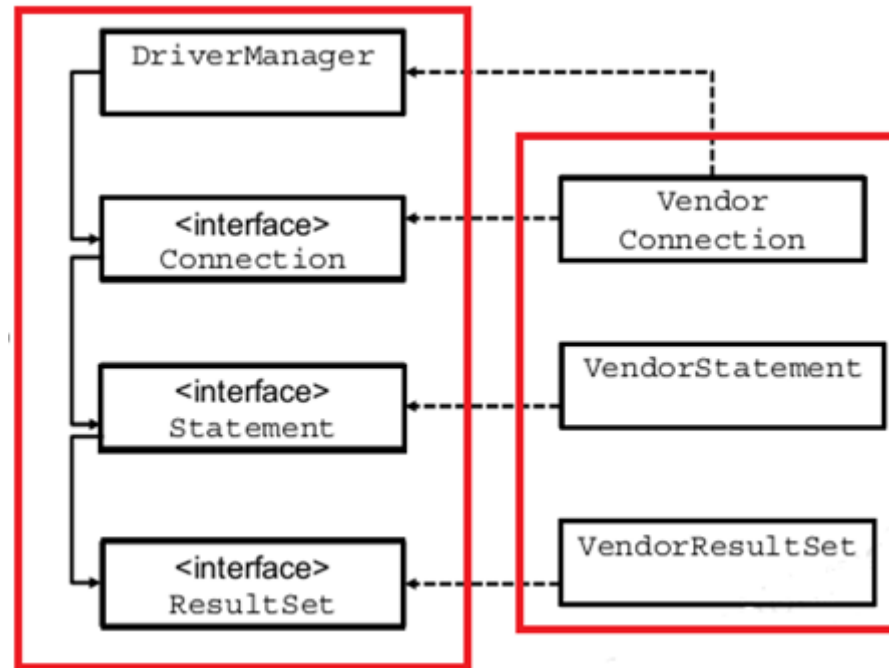
- There are **four types of JDBC drivers**
 1. JDBC-ODBC Bridge Driver (**Deprecated in Java 8 and later**)
 2. Native Driver
 3. Network Protocol Driver
 4. Thin Driver (**Most Commonly Used**)

Java Database Connectivity (JDBC)

Steps to Connect to the Database

There are **5 steps** to connect any java application with the database using JDBC

- Register the Driver class
- Create connection
- Create statement
- Execute queries
- Close connection



Java Database Connectivity (JDBC)

Steps to Connect to the Database

- The JDBC **DriverManager** class defines objects which can **connect Java applications to a JDBC driver**.
- DriverManager is considered the **backbone** of JDBC architecture.
- DriverManager class **manages the JDBC drivers** that are installed on the system.
- Its **getConnection() method** is used to establish a connection to a database.

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Steps 1) Register the driver class

- To load the driver or register it before using it in the program. There should be registration once in program. We can register a driver in any of the **two ways**:
 1. Using **forName()** method
 2. Using **registerDriver()** method
- 1. The **forName()** method of Class is used to **register the driver class**. This method is used to **dynamically load** the driver class. Once loaded, the Driver class creates an instance of itself.

Steps to Connect to the Database

Syntax:

```
public static void forName(String className)throws ClassNotFoundException
```

Example: Java program to **load oracle driver** to establish database connection

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Java Database Connectivity (JDBC)

Steps to Connect to the Database

2. Using **registerDriver()** method

- DriverManager is an inbuilt class of Java that comes with a static member register.
- Need to call the drivers class' constructor at compile-time .

Syntax:

```
public static void registerDriver(Driver driver) throws SQLException
```

Example: Java program to **load oracle driver** to establish database connection

```
DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());
```

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Steps 2) Create the connection object

- The **getConnection()** method of **DriverManager** class is used to **establish connection** with the database
- Uses a **username, password, and a jdbc url** to establish a connection to the database
- It returns a **connection object**.
- A jdbc Connection represents a **session/connection** with a specific database.
- Within the context of a Connection, the statements are **executed and results** are returned.
- An application can have **one or more** connections with a single database, or it can have **many connections** with different databases.

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Syntax:

```
public static Connection getConnection(String url) throws SQLException  
public static Connection getConnection(String url,String name,String password)  
throws SQLException
```

Example:

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb",  
"root","password");
```

jdbc → the API (Java Database Connectivity).

mysql → the database we are connecting to.

localhost → the server name (can also use IP address, e.g., 127.0.0.1).

3306 → the default MySQL port number.

testdb → the database/schema name inside MySQL.

"root" → MySQL username,"password" → MySQL password.

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Steps 3) Create the statement object

- The `createStatement()` method of **Connection interface** is used to create statement.
- The object of statement is responsible to execute queries with the database.
- Can execute Statement objects, and they generate ResultSet objects, which is a table of data representing a database result set.
- Need a Connection object to create a Statement object

Syntax:

```
public Statement createStatement() throws SQLException
```

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Example:

```
Statement stmt=con.createStatement();
```

JDBC represents **statements** using one of the following classes:

Statement:

- Used to implement simple SQL statements with no parameters.

PreparedStatement: (Extends Statement.)

- The statement is cached and then the execution path is pre-determined on the database server allowing it to be executed **multiple times** in an efficient manner.

Java Database Connectivity (JDBC)

Steps to Connect to the Database

CallableStatement: (Extends PreparedStatement.)

- Used to execute **stored procedures** that may contain both input and output parameters

Steps 4) Execute the Query

- The **executeQuery()** method of Statement interface is used to execute queries to the database.
- This method **returns the object** of ResultSet that can be used to get all the records of a table.

Syntax:

```
public ResultSet executeQuery(String sql)throws SQLException
```

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Example:

```
ResultSet rs=stmt.executeQuery("select * from employee");

while(rs.next()){
    System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Steps 4) Execute the Query

To execute a query, call an **execute method from Statement** such as the following:

execute:

- Returns **true** if the first object that the query returns is a ResultSet object.
- Use this method if the query could return one or more ResultSet objects.
- Retrieve the ResultSet objects returned from the query by repeatedly calling Statement.getResultSet.

executeQuery:

- Returns one ResultSet object.

executeUpdate:

- Returns an integer representing the **number of rows** affected by the SQL statement. Use this method if you are using **INSERT,DELETE, or UPDATE SQL** statements

Java Database Connectivity (JDBC)

Steps to Connect to the Database

Steps 5) Close the connection object

- The **close()** method of Connection interface is used to **close the connection**
- By **closing** connection object statement and ResultSet will be **closed automatically**.

Syntax:

```
public void close() throws SQLException
```

Example:

```
con.close();
```

Java Database Connectivity

JDBC with MySQL

Information required about MySQL database as follows:

Driver class:

The driver class for the MySQL database is **`com.mysql.cj.jdbc.Driver`**

Connection URL:

The connection URL for the MySQL database is **`jdbc:mysql://localhost:3306/databasename`**

jdbc → the API

mysql → the database

localhost → the server name on which MySQL is running (you can also use IP address)

3306 → the default port number of MySQL

databasename → the name of the MySQL database you want to connect to

Username:

Default username for MySQL is usually **root**.

Password:

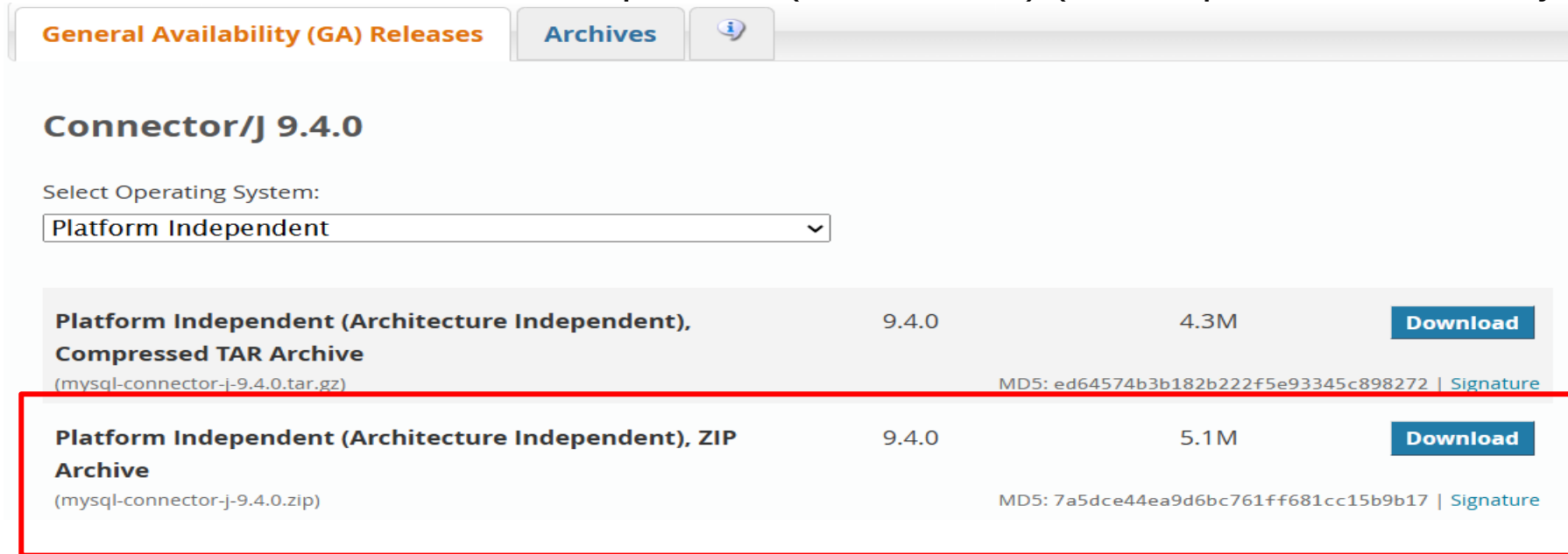
The password is set by the user at the time of installing MySQL (commonly blank by default on local setups).

Java Database Connectivity (JDBC)

Configuring a JDBC development environment (MySQL JDBC Driver)

Step 1: Download MySQL Connector/J

- Open your browser and go to the **official download page**: <https://dev.mysql.com/downloads/connector/j/>
- On that page:
 - Scroll down to “**MySQL Connector/J**”.
 - Choose the latest Platform Independent (**ZIP Archive**). (Do not pick MSI or RPM; just get the ZIP)



The screenshot shows the MySQL Connector/J download page. At the top, there are tabs for "General Availability (GA) Releases" (selected), "Archives", and an information icon. Below the tabs, the version "Connector/J 9.4.0" is displayed. A dropdown menu for "Select Operating System:" is set to "Platform Independent". Below this, there is a table of download options. The second row, "Platform Independent (Architecture Independent), ZIP Archive", is highlighted with a red box. This row shows version 9.4.0, a size of 5.1M, and a "Download" button. The first row, "Platform Independent (Architecture Independent), Compressed TAR Archive", is not highlighted. Both rows include MD5 checksums and links to signatures.

Download Option	Version	Size	Action
Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-j-9.4.0.tar.gz)	9.4.0	4.3M	Download
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-9.4.0.zip)	9.4.0	5.1M	Download

Java Database Connectivity (JDBC)

Configuring a JDBC development environment (MySQL JDBC Driver)

- **Click Download.**
 - Oracle might ask you to log in or sign up.
 - Skip Login → There is usually a link: “No thanks, just start my download.”
- **After download finishes, you’ll have a file like:**
 - mysql-connector-j-9.4.0.zip (version may differ).

Java Database Connectivity (JDBC)

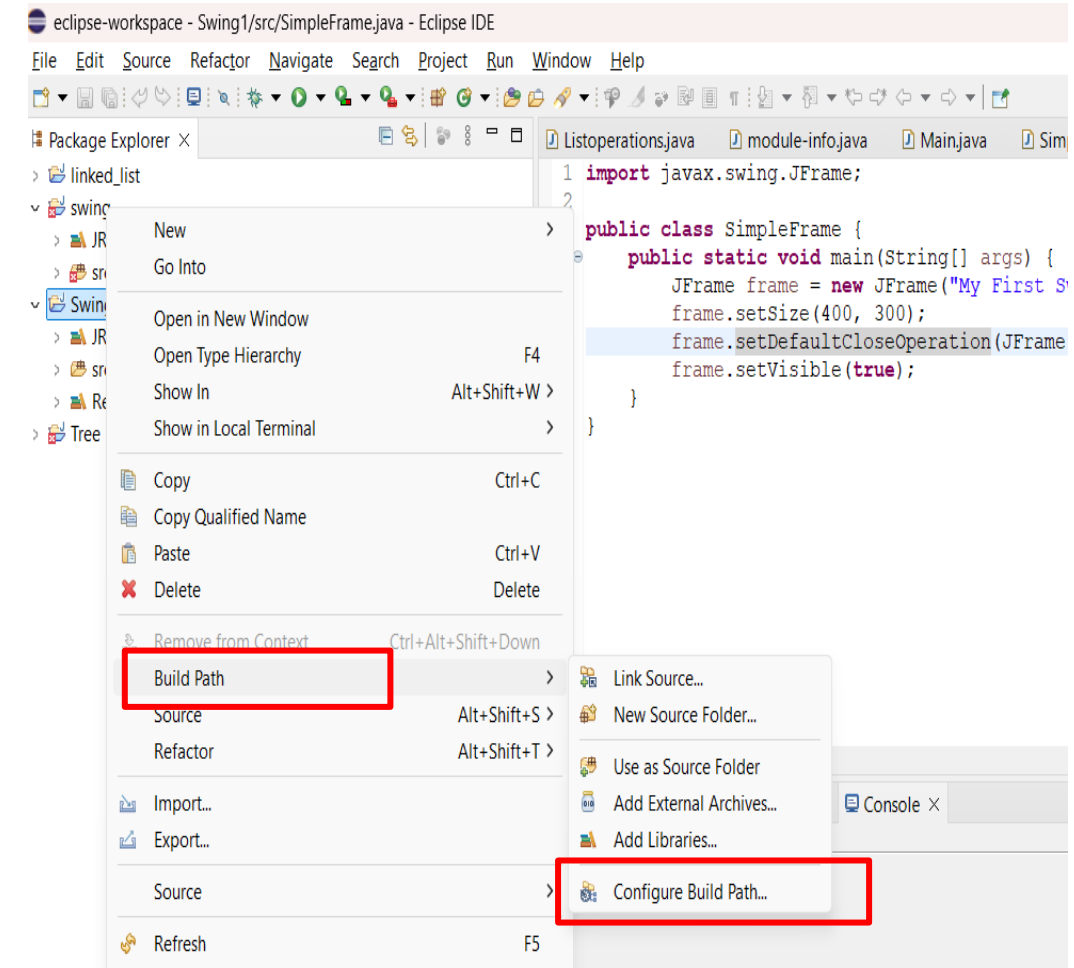
Configuring a JDBC development environment (MySQL JDBC Driver)

Step 2: Extract the ZIP

- Go to your Downloads folder.
- Right-click → Extract All (or use WinRAR/7zip).
- Inside the extracted folder, you'll see files like:
- mysql-connector-j-9.4.0.jar

Step 3: Add JAR to Eclipse Project

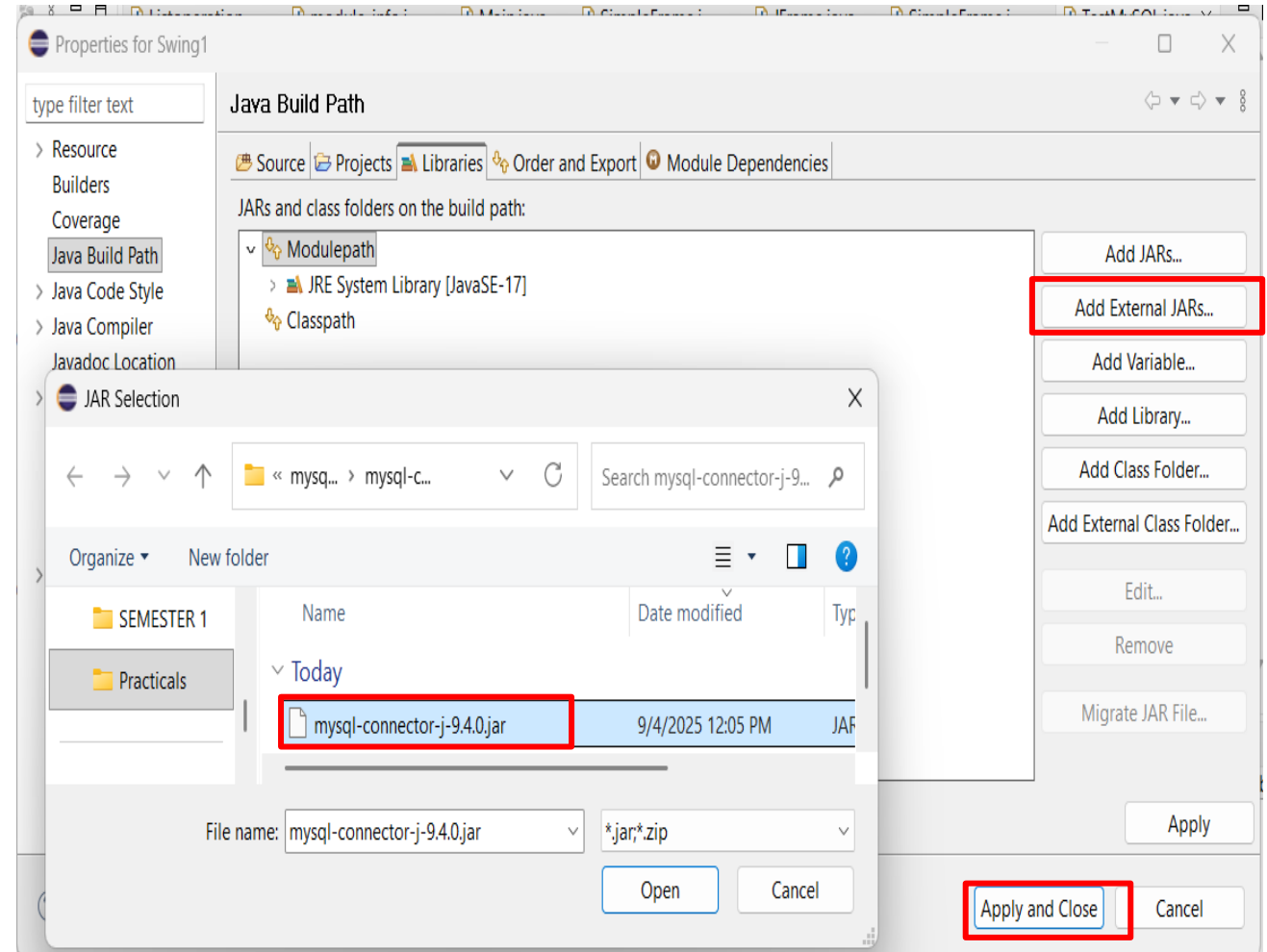
- Open Eclipse.
- In Package Explorer, right-click **your project** → **Build Path** → **Configure Build Path...**
- Go to the **Libraries** tab.
- Click **Add External JARs...**



Java Database Connectivity (JDBC)

Configuring a JDBC development environment (MySQL JDBC Driver)

- Browse to the folder where you extracted MySQL Connector/J.
- **Example:**
C:\Users\YourName\Downloads\mysql-connector-j-9.4.0\
- Select the file: mysql-connector-j-9.4.0.jar
- Click **Apply and Close**.

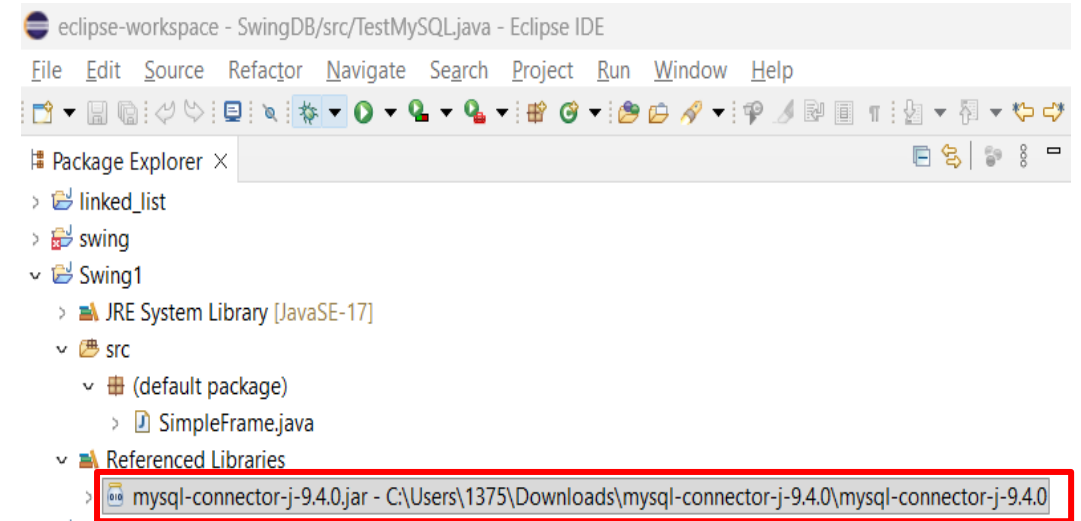


Java Database Connectivity (JDBC)

Configuring a JDBC development environment (MySQL JDBC Driver)

Step 4: Verify in Eclipse

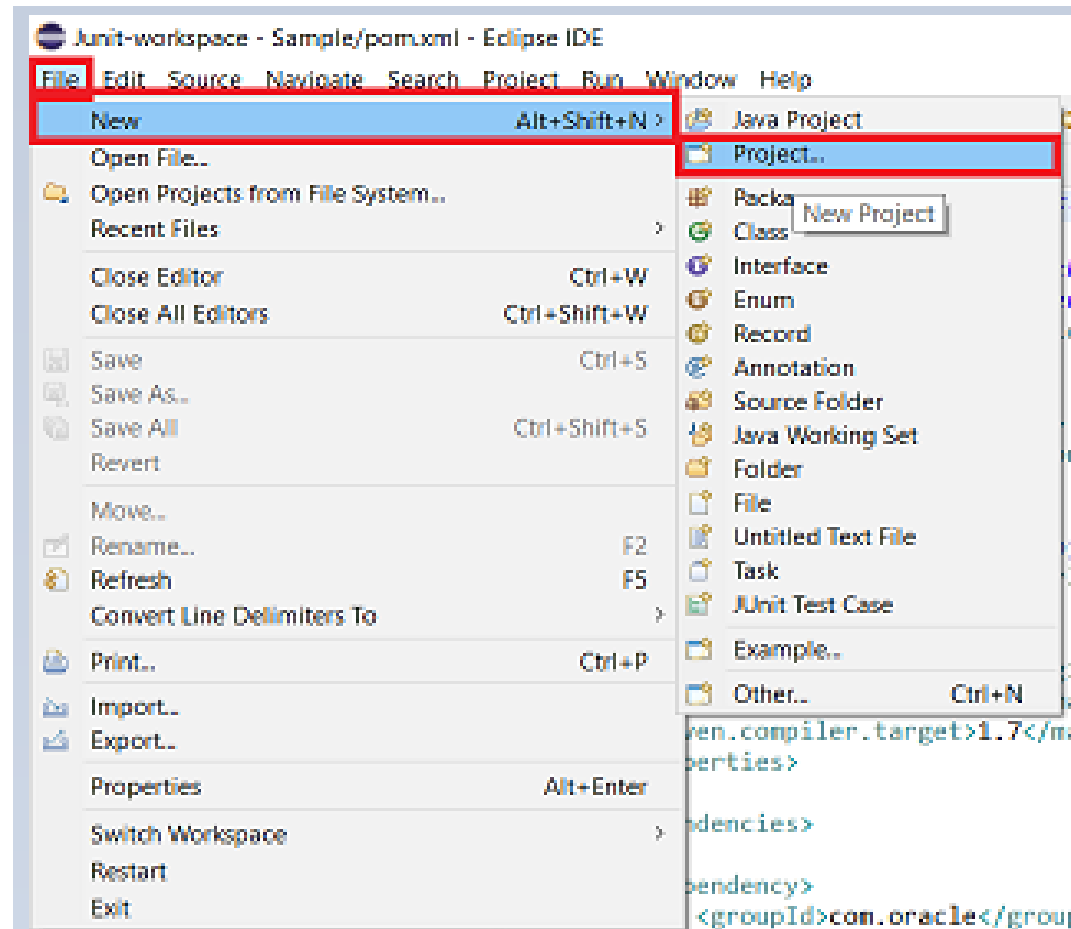
- In your project, expand **Referenced Libraries**.
- You should now see:
 - mysql-connector-j-9.4.0.jar
- This means Eclipse can now use the MySQL driver.



Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)

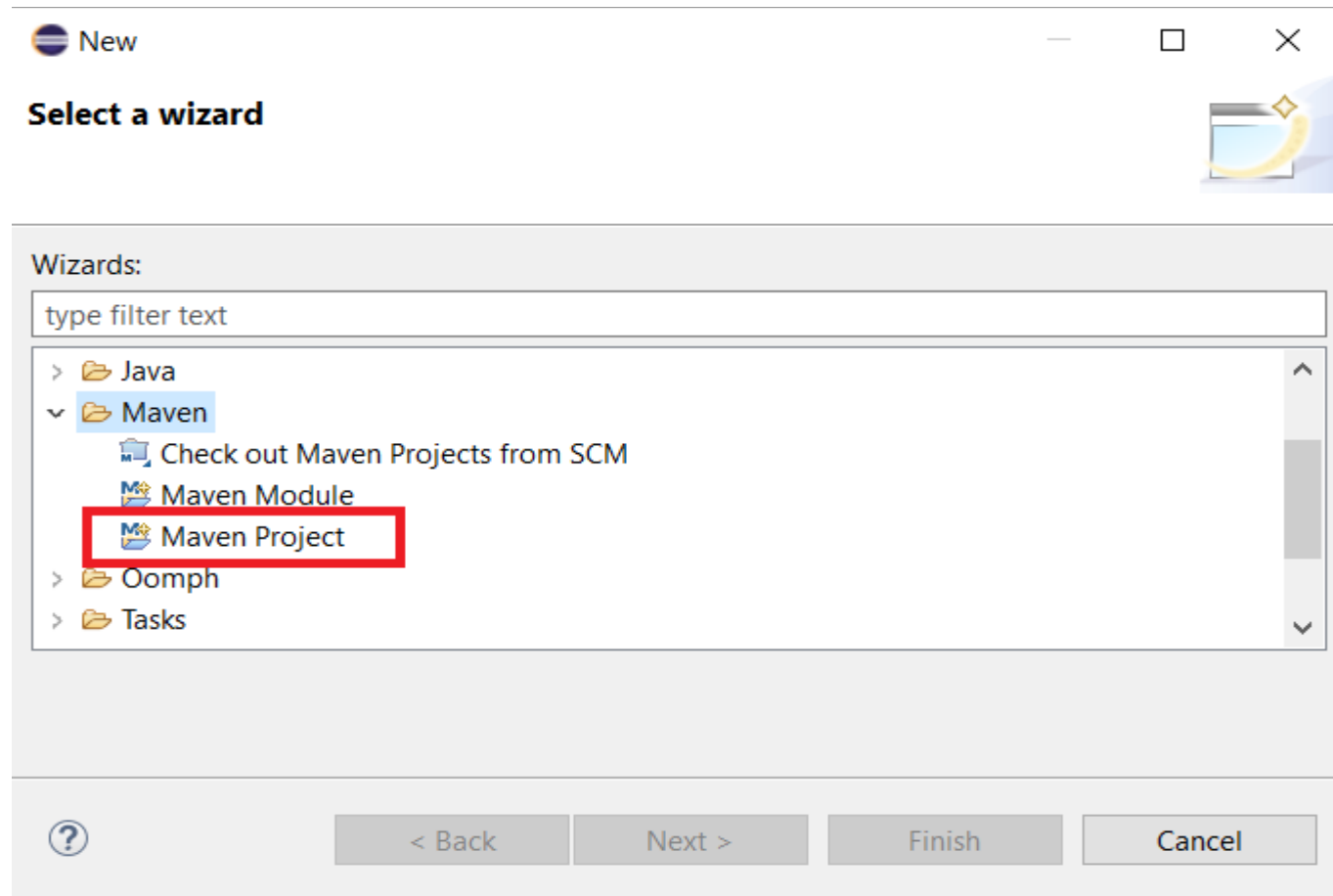
- Open Eclipse File → New → Project



Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)

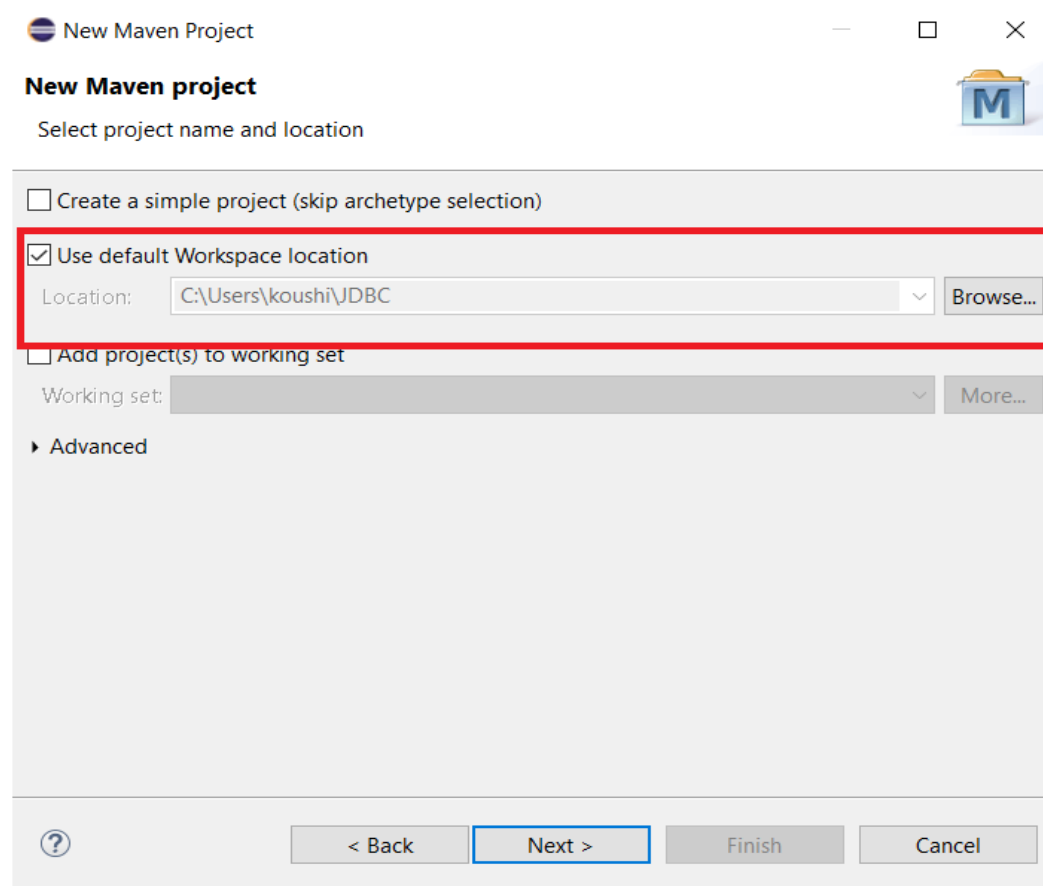
- Select “Maven Project” and Click “Next”



Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)

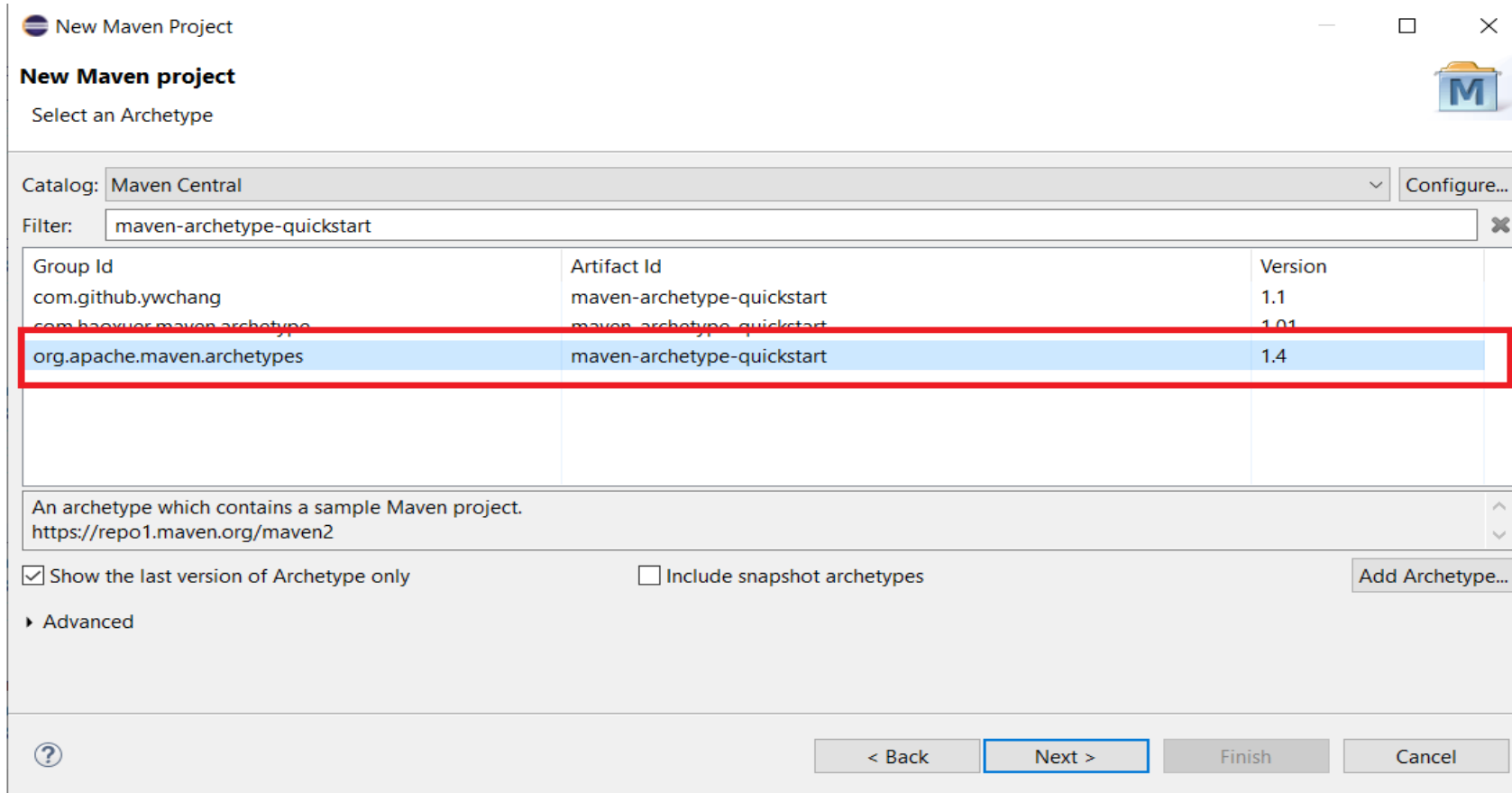
- Choose the workspace and Click “Next”



Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)

- Choose the archetype and Click “Next”



New Maven Project

New Maven project
Select an Archetype

Catalog: Maven Central Configure...

Filter: maven-archetype-quickstart ×

Group Id	Artifact Id	Version
com.github.ywchang	maven-archetype-quickstart	1.1
com.baeruer.maven.archetype	maven-archetype-quickstart	1.01
org.apache.maven.archetypes	maven-archetype-quickstart	1.4

An archetype which contains a sample Maven project.
<https://repo1.maven.org/maven2>

☒ Show the last version of Archetype only ☐ Include snapshot archetypes Add Archetype...

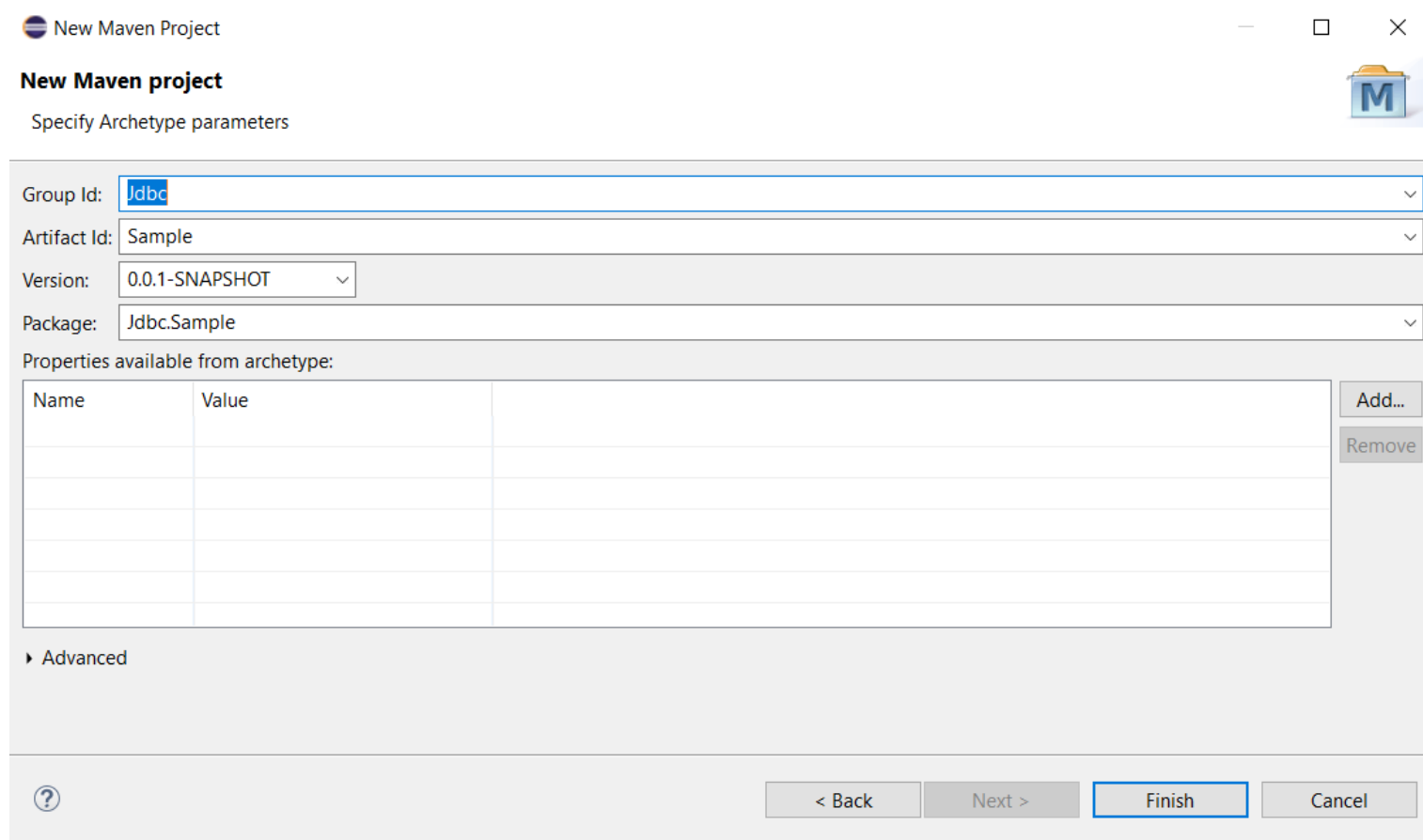
► Advanced

? < Back Next > Finish Cancel

Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)

- Enter Group ID and Artifact id ...Package will be named automatically. Click Finish



New Maven Project

New Maven project

Specify Archetype parameters

Group Id: jdbc

Artifact Id: Sample

Version: 0.0.1-SNAPSHOT

Package: jdbc.Sample

Properties available from archetype:

Name	Value

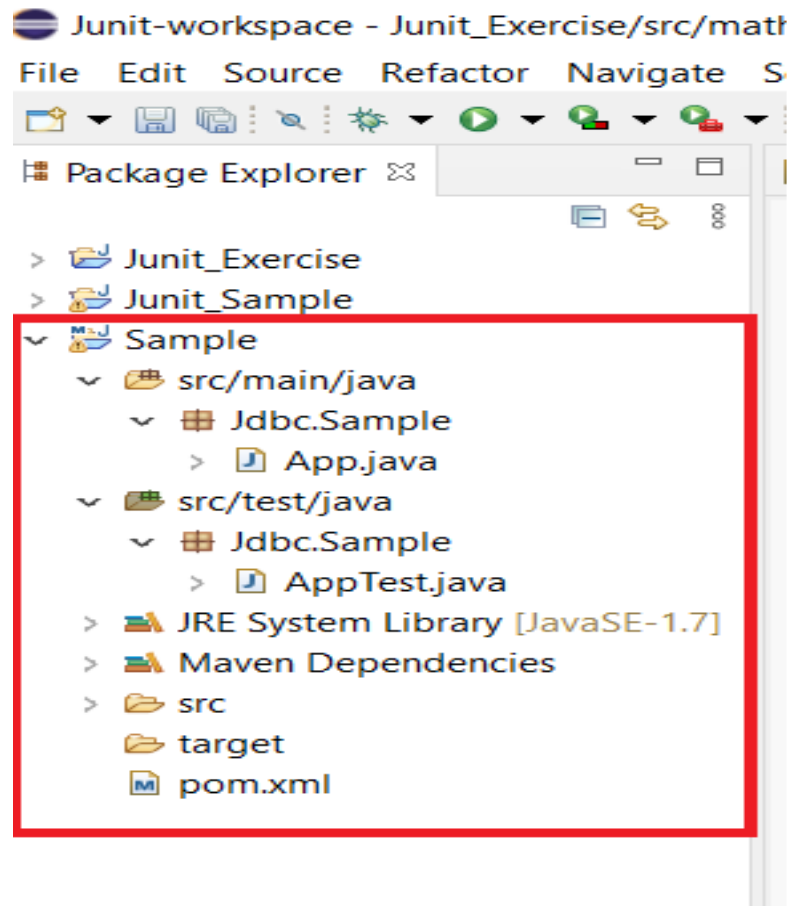
Advanced

< Back Next > Finish Cancel

Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)

- Maven project was built



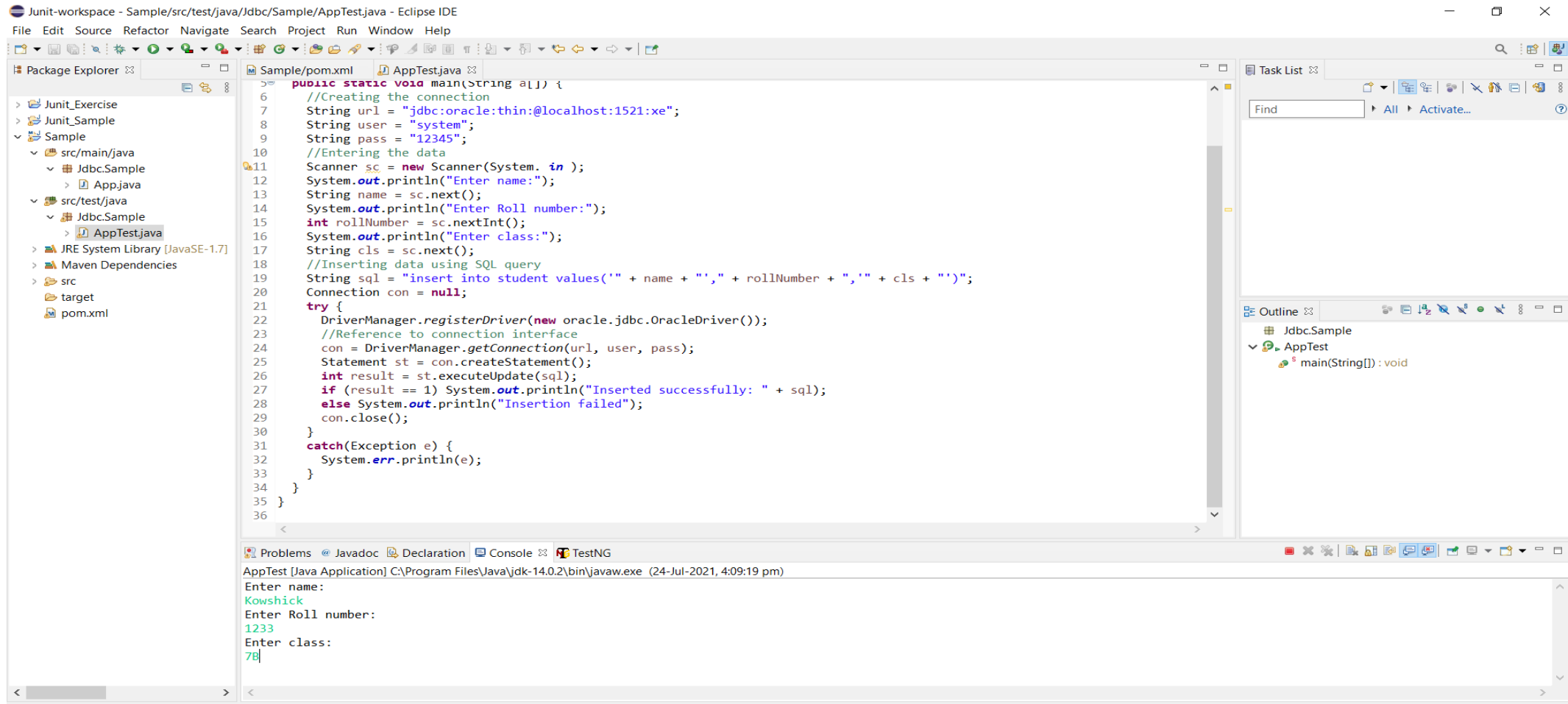
Configuring a JDBC development environment (Using Maven)

- **Add the dependencies in “pom.xml”**

```
<!-- https://mvnrepository.com/artifact/com.mysql/mysql-connector-j -->  
<dependency>  
    <groupId>com.mysql</groupId>  
    <artifactId>mysql-connector-j</artifactId>  
    <version>9.4.0</version>  
</dependency>
```

Java Database Connectivity (JDBC)

Configuring a JDBC development environment (Using Maven)



The screenshot shows the Eclipse IDE with the following components:

- Package Explorer:** Shows the project structure: Junit-Exercise, Junit_Sample, Sample (src/main/java, src/test/java), JRE System Library [JavaSE-1.7], Maven Dependencies, src, target, and pom.xml.
- Editor:** Displays the code for `AppTest.java`. The code is as follows:


```

1 public static void main(String a[]) {
2     //Creating the connection
3     String url = "jdbc:oracle:thin:@localhost:1521:xe";
4     String user = "system";
5     String pass = "12345";
6     //Entering the data
7     Scanner sc = new Scanner(System.in);
8     System.out.println("Enter name:");
9     String name = sc.next();
10    System.out.println("Enter Roll number:");
11    int rollNumber = sc.nextInt();
12    System.out.println("Enter class:");
13    String cls = sc.next();
14    //Inserting data using SQL query
15    String sql = "insert into student values('" + name + "', " + rollNumber + ", '" + cls + "')";
16    Connection con = null;
17    try {
18        DriverManager.registerDriver(new oracle.jdbc.OracleDriver());
19        //Reference to connection interface
20        con = DriverManager.getConnection(url, user, pass);
21        Statement st = con.createStatement();
22        int result = st.executeUpdate(sql);
23        if (result == 1) System.out.println("Inserted successfully: " + sql);
24        else System.out.println("Insertion failed");
25        con.close();
26    }
27    catch (Exception e) {
28        System.err.println(e);
29    }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
      
```
- Task List:** Empty.
- Outline:** Shows the class hierarchy: Jdbc.Sample > AppTest > main(String[]): void.
- Console:** Shows the output of the application:


```

AppTest [Java Application] C:\Program Files\Java\jdk-14.0.2\bin\javaw.exe (24-Jul-2021, 4:09:19 pm)
Enter name:
Kowshick
Enter Roll number:
1233
Enter class:
7B
      
```

Java Database Connectivity (JDBC)

Example #1

Table creation

```
CREATE TABLE student (  
    Id int(11) NOT NULL,  
    Name varchar(45) NOT NULL,  
    Course varchar(45) NOT NULL,  
    Email varchar(45) NOT NULL,  
    PRIMARY KEY (Id));
```

Java Database Connectivity (JDBC)

Example #1 (Create table)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class JdbcCreateTableDemo {
    public static void main(String[] args) {

        String dbURL = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "Admin";

        try (Connection conn = DriverManager.getConnection(dbURL, username, password)) {

            String sql = "CREATE TABLE Student " +
                "(ID INT NOT NULL, " +
                "Name VARCHAR(255), " +
                "Course VARCHAR(255), " +
                "Email VARCHAR(255), " +
                "PRIMARY KEY (ID))";

            PreparedStatement ps = conn.prepareStatement(sql);

            ps.executeUpdate();
            System.out.println("Table created successfully using PreparedStatement!");

        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```

Java Database Connectivity (JDBC)

Example #1 (Insert)

Inserting a new record into the table (101,"Bibin","Maths","bibin@gmail.com")

```
public class JdbcInsertDemo {
    public static void main(String[] args) {

        String dbURL = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "Admin";

        try (Connection conn = DriverManager.getConnection(dbURL, username, password)) {

            String sql = "INSERT INTO Student (Id, Name, Course, Email) VALUES (?, ?, ?, ?)";

            PreparedStatement statement = conn.prepareStatement(sql);

            statement.setInt(1, 101);
            statement.setString(2, "Bibin");
            statement.setString(3, "Maths");
            statement.setString(4, "bibin@gmail.com");

            int rowsInserted = statement.executeUpdate();

            if (rowsInserted > 0) {
                System.out.println("A new user was inserted successfully!");
            }

        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```

Java Database Connectivity (JDBC)

Example #1 (Select)

Displaying all the records from table

```
import java.sql.ResultSet;

public class JdbcSelectStudentDemo {
    public static void main(String[] args) {
        String dbURL = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "Admin";

        try (Connection conn = DriverManager.getConnection(dbURL, username, password)) {
            String sql = "SELECT * FROM Student";

            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery();
            int count = 0;

            while (rs.next()) {
                int id = rs.getInt("ID");
                String name = rs.getString("Name");
                String course = rs.getString("Course");
                String email = rs.getString("Email");

                String output = "Student #%d: %d - %s - %s - %s";
                System.out.println(String.format(output, ++count, id, name, course, email));
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```


Java Database Connectivity (JDBC)

Example #1 (Update)

Updating the course to “Physics” for the name “Bibin”

```
public class JdbcUpdateDemo {  
  
    public static void main(String[] args) {  
        String dbURL = "jdbc:mysql://localhost:3306/testdb";  
        String username = "root";  
        String password = "Admin";  
  
        try (Connection conn = DriverManager.getConnection(dbURL, username, password)) {  
            String sql = "UPDATE Student SET Course = ?, Email = ? WHERE Name = ?";  
            PreparedStatement statement = conn.prepareStatement(sql);  
  
            statement.setString(1, "Physics");  
            statement.setString(2, "bibin6420@gmail.com");  
            statement.setString(3, "Bibin");  
  
            int rowsUpdated = statement.executeUpdate();  
            if (rowsUpdated > 0) {  
                System.out.println("Student record updated successfully!");  
            } else {  
                System.out.println("No matching student found!");  
            }  
        } catch (SQLException ex) {  
            ex.printStackTrace();  
        }  
    }  
}
```

Java Database Connectivity (JDBC)

Example #1 (Delete)

Deleting the record from table whose name is “Bibin”

```
public class JdbcDeleteDemo {
    public static void main(String[] args) {
        String dbURL = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "secret";

        try (Connection conn = DriverManager.getConnection(dbURL, username, password)) {

            String sql = "DELETE FROM Student WHERE Name = ?";
            PreparedStatement statement = conn.prepareStatement(sql);
            statement.setString(1, "Bibin");

            int rowsDeleted = statement.executeUpdate();
            if (rowsDeleted > 0) {
                System.out.println("Student record deleted successfully!");
            } else {
                System.out.println("No matching student found!");
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```

Java Database Connectivity (JDBC)

Example #2

Table creation – MySQL WorkBench

Create Employee Table with E_ID, Name and Department;

MySQL Query:

```
CREATE TABLE employee (  
  
E_Id INT PRIMARY KEY,  
  
Name VARCHAR(40),  
  
Dept VARCHAR(40)  
  
);
```

Java Database Connectivity (JDBC)

Example #2

Procedure Creation – MySQL WorkBench

1. Create Procedure insert_employee()

DELIMITER \$\$

CREATE PROCEDURE insert_employee(

IN p_id INT,

IN p_name VARCHAR(40),

IN p_dept VARCHAR(40)

)

BEGIN

INSERT INTO employee (E_Id, Name, Dept)

VALUES (p_id, p_name, p_dept);

END \$\$

DELIMITER ;

**NOTE: To verify successful procedure creation
SHOW PROCEDURE STATUS WHERE Db = 'testdb';**

Java Database Connectivity (JDBC)

Example #2

2. Create Procedure get_all_employees()

DELIMITER \$\$

CREATE PROCEDURE get_all_employees()

BEGIN

SELECT * FROM employee;

END \$\$

DELIMITER ;

Java Database Connectivity (JDBC)

Example #2

3. Create Procedure update_employee()

DELIMITER \$\$

CREATE PROCEDURE update_employee(

IN p_id INT,

IN p_dept VARCHAR(40)

)

BEGIN

UPDATE employee SET Dept = p_dept

WHERE E_Id = p_id;

END \$\$

DELIMITER ;

Java Database Connectivity (JDBC)

Example #2

4. Create Procedure delete_employee()

DELIMITER \$\$

CREATE PROCEDURE delete_employee(

IN p_id INT

)

BEGIN

DELETE FROM employee

WHERE E_Id = p_id;

END \$\$

DELIMITER ;

Java Database Connectivity (JDBC)

Example #2

5. Create Procedure get_employee_by_id()

DELIMITER \$\$

CREATE PROCEDURE get_employee_by_id(

IN p_id INT,

OUT o_name VARCHAR(40),

OUT o_dept VARCHAR(40)

)

BEGIN

SELECT Name, Dept INTO o_name, o_dept

FROM employee

WHERE E_Id = p_id;

END \$\$

DELIMITER ;

Java Database Connectivity (JDBC)

Example #2

Procedures Created Successfully!

86 • `SHOW PROCEDURE STATUS WHERE Db = 'testdb';`

Result Grid | Filter Rows: | Export: | Wrap Cells

	Db	Name	Type	Language	Definer
▶	testdb	delete_employee	PROCEDURE	SQL	root@localhost
	testdb	get_all_employees	PROCEDURE	SQL	root@localhost
	testdb	get_employee_by_id	PROCEDURE	SQL	root@localhost
	testdb	insert_employee	PROCEDURE	SQL	root@localhost

Result Grid | Form Editor

Result 1 x | Read Only | Context Help | Snippets

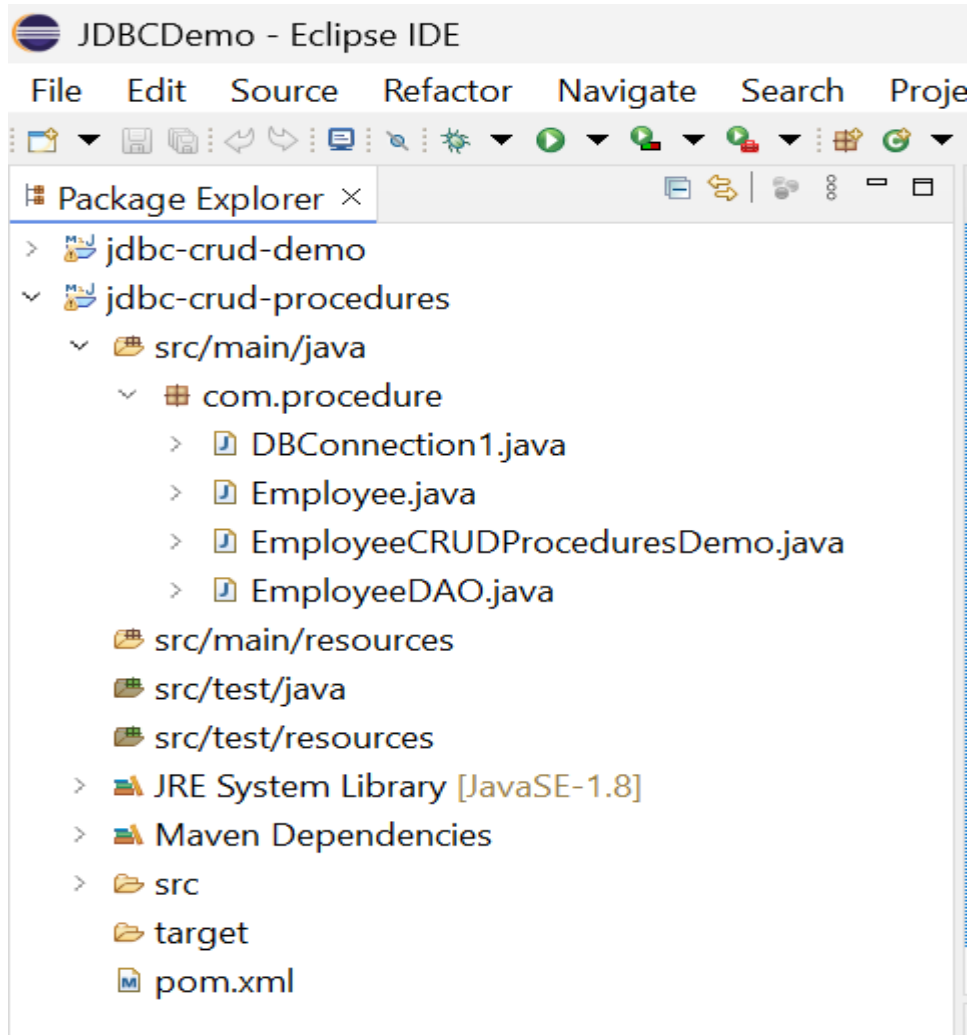
Output

Action Output

#	Time	Action	Message
1	16:24:32	SHOW PROCEDURE STATUS WHERE Db = 'testdb'	5 row(s) returned

Java Database Connectivity (JDBC)

Example #2 – Maven Project Creation



Example #2 (Stored Procedure) - DBConnection1.java

```
package com.procedure;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection1 {

    // Method to get a database connection
    public static Connection getConnection() throws SQLException {
        // MySQL URL, username and password
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "Admin";

        // Return a connection object
        return DriverManager.getConnection(url, user, password);
    }
}
```

Example #2 (Stored Procedure) - Employee.java

```
package com.procedure;

public class Employee {
    private int id;
    private String name;
    private String dept;

    public Employee(int id, String name, String dept) {
        this.id = id;
        this.name = name;
        this.dept = dept;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDept() { return dept; }
}
```

Example #2 (Stored Procedure) - EmployeeDAO.java

```
package com.procedure;

import java.sql.*;

public class EmployeeDAO {

    public void insertEmployee(Employee emp) {
        try (Connection conn = DBConnection1.getConnection();
            CallableStatement cs = conn.prepareCall("{call insert_employee(?,?,?)}")) {

            cs.setInt(1, emp.getId());
            cs.setString(2, emp.getName());
            cs.setString(3, emp.getDept());

            cs.execute();
            System.out.println("Inserted!");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Example #2 (Stored Procedure) - EmployeeDAO.java

```
public void displayEmployees() {  
    try (Connection conn = DBConnection1.getConnection();  
        CallableStatement cs = conn.prepareCall("{call get_all_employees()}");  
        ResultSet rs = cs.executeQuery()) {  
  
        while (rs.next()) {  
            System.out.println(  
                rs.getInt("E_Id") + " | " +  
                rs.getString("Name") + " | " +  
                rs.getString("Dept")  
            );  
        }  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

Example #2 (Stored Procedure) - EmployeeDAO.java

```
public void updateEmployee(int id, String dept) {  
    try (Connection conn = DBConnection1.getConnection();  
        CallableStatement cs = conn.prepareCall("{call update_employee(?,?)}")) {  
  
        cs.setInt(1, id);  
        cs.setString(2, dept);  
  
        cs.execute();  
        System.out.println("Updated!");  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

Example #2 (Stored Procedure) - EmployeeDAO.java

```
public void deleteEmployee(int id) {  
    try (Connection conn = DBConnection1.getConnection();  
        CallableStatement cs = conn.prepareCall("{call delete_employee(?)}}")) {  
  
        cs.setInt(1, id);  
  
        cs.execute();  
        System.out.println("Deleted!");  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```


Example #2 (Stored Procedure) - EmployeeDAO.java

```
public void getEmployeeById(int id) {  
    try (Connection conn = DBConnection1.getConnection();  
        CallableStatement cs = conn.prepareCall("{call get_employee_by_id(?,?,?)}")) {  
  
        cs.setInt(1, id);  
        cs.registerOutParameter(2, Types.VARCHAR);  
        cs.registerOutParameter(3, Types.VARCHAR);  
  
        cs.execute();  
  
        System.out.println("Name: " + cs.getString(2));  
        System.out.println("Dept: " + cs.getString(3));  
  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}
```

Example #2 (Stored Procedure) - EmployeeCRUDProceduresDemo.java

```
package com.procedure;

import java.util.Scanner;

public class EmployeeCRUDProceduresDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        EmployeeDAO dao = new EmployeeDAO();
        int choice;

        do {
            System.out.println("\nPlease choose an option from the menu:");
            System.out.println("1. Insert a new employee");
            System.out.println("2. Display all employees");
            System.out.println("3. Update an employee's department");
            System.out.println("4. Delete an employee");
            System.out.println("5. Get employee details by ID");
            System.out.println("0. Exit");
```

Example #2 (Stored Procedure) - EmployeeCRUDProceduresDemo.java

```
System.out.print("Enter your choice (0-5): ");
choice = sc.nextInt();
sc.nextLine(); // consume newline

switch (choice) {

    case 1:
        System.out.println("\n--- Insert Employee ---");
        System.out.print("Enter Employee ID: ");
        int id = sc.nextInt();
        sc.nextLine(); // consume newline

        System.out.print("Enter Employee Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Employee Department: ");
        String dept = sc.nextLine();

        dao.insertEmployee(new Employee(id, name, dept));
        break;
```

Example #2 (Stored Procedure) - EmployeeCRUDProceduresDemo.java

case 2:

```
System.out.println("\n--- Employee List ---");  
dao.displayEmployees();  
break;
```

case 3:

```
System.out.println("\n--- Update Employee Department ---");  
System.out.print("Enter Employee ID to update: ");  
id = sc.nextInt();  
sc.nextLine(); // consume newline
```

```
System.out.print("Enter new Department: ");  
dept = sc.nextLine();
```

```
dao.updateEmployee(id, dept);  
break;
```

Example #2 (Stored Procedure) - EmployeeCRUDProceduresDemo.java

case 4:

```
System.out.println("\n--- Delete Employee ---");  
System.out.print("Enter Employee ID to delete: ");  
id = sc.nextInt();  
dao.deleteEmployee(id);  
break;
```

case 5:

```
System.out.println("\n--- Get Employee By ID ---");  
System.out.print("Enter Employee ID: ");  
id = sc.nextInt();  
dao.getEmployeeById(id);  
break;
```

Example #2 (Stored Procedure) - EmployeeCRUDProceduresDemo.java

```
        case 0:
            System.out.println("\nExiting Employee Management System. Goodbye!");
            break;

        default:
            System.out.println("\nInvalid choice! Please enter a number between 0 and 5.");
    }

    } while (choice != 0);

    sc.close();
}
}
```

Java Database Connectivity (JDBC)

Quiz



1. Select the packages in which JDBC classes are defined?

a) java.sql and javax.sql

b) rdb and javax.rdb

c) jdbc and java.jdbc.sql

d) jdbc and javax.jdbc

a) java.sql and javax.sql

Quiz



2. Which of the following method is used to perform DML statements in JDBC?

a) executeResult()

b) execute()

c) executeQuery()

d) executeUpdate()

d) executeUpdate()

Quiz



3. Which of the following is not a valid statement in JDBC?

a) Statement

b) QueryStatement

c) PreparedStatement

d) CallableStatement

b) QueryStatement

Quiz



4. Which is used to call the stored procedures and functions?

a) CallableStatement
Interface

b) PreparedStatement
Interface

c) Both

d) None

a) CallableStatement Interface

Quiz



5. The `ResultSet.next` method is used to move to the next row of the `ResultSet`, making it the current row.

a) True

b) False

a) True

THANK YOU